Week 3: A Simple Spam Workflow

Capture, parse, filter, classify,
then act with confirmation

AI personal assistant in Python

How to approach this week

Steps A–E are your guide: capture, parse, filter, classify, confirm

PATH 1: BUILD WITH AN AI AGENT

Use the steps to specify what to build.
Understand and test each part.
Parse saved data with Pydantic.
Structure model inputs and outputs.

PATH 2: START WITH OUR CODE

Pull the week-3 branch.
Read the code and trace the workflow.
Run it and test each step.
Modify it for your own email service.

YOUR TASK: BUILD BEYOND SPAM

Choose what you want your email assistant to do.
Explore Nylas: draft replies or download attachments for image processing.
Keep user confirmation before consequential actions.

Start: app/workflows/spam_workflow.py; app/main.py; week3/fastapi-tutorial/

Week 2 recap vs Week 3 goals

WEEK 2

Nylas connected
FastAPI receives webhooks
Raw email JSON saved
Small LLM calls tested

WEEK 3

Decide: spam or not spam
Delete spam after confirmation
Learn FastAPI and do the exercise
in week3/fastapi-tutorial

SUCCESS CRITERIA

Capture 3 JSON examples and validate one with Pydantic

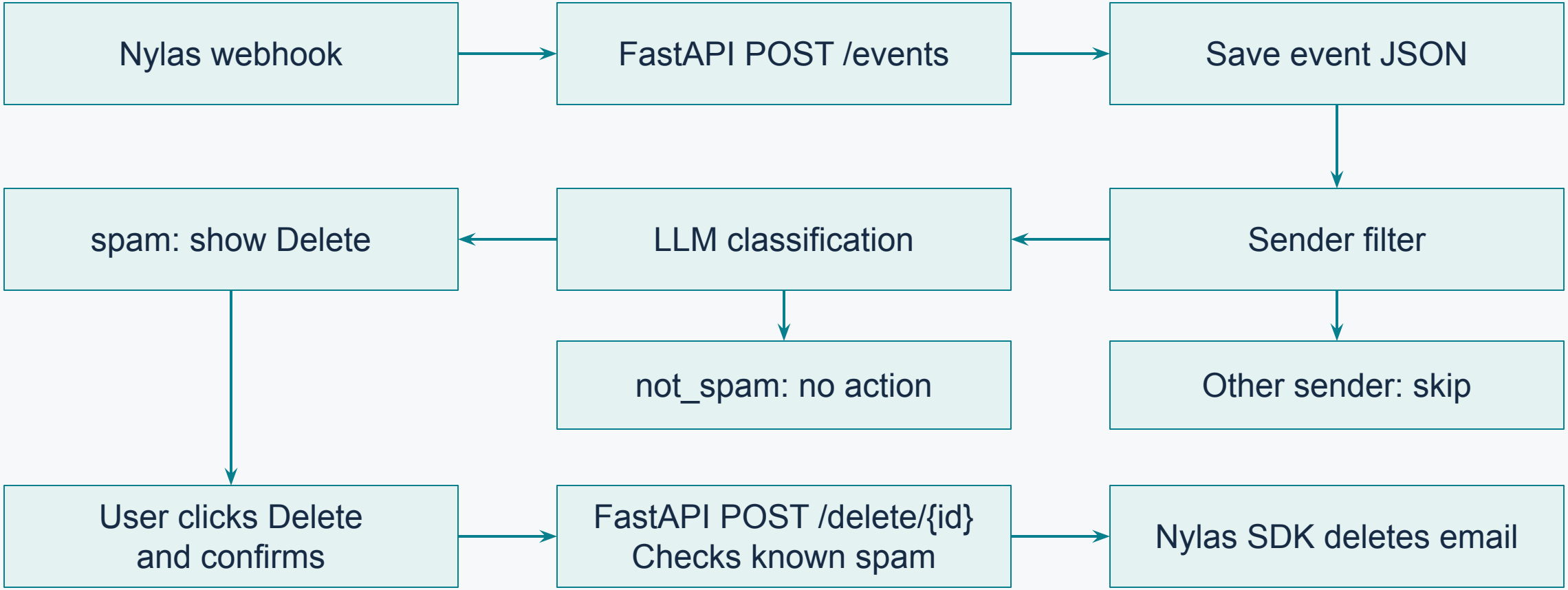
Filter your own sender, classify with confidence, confirm deletion

Complete the FastAPI tutorial and send a test request

Code: app/main.py; app/workflows/spam_workflow.py; week3/fastapi-tutorial/

FastAPI receives events and handles actions

Week 2: capture incoming email. Week 3: classify and confirm actions.



Code: app/main.py: check_spam(), delete(); app/workflows/spam_workflow.py

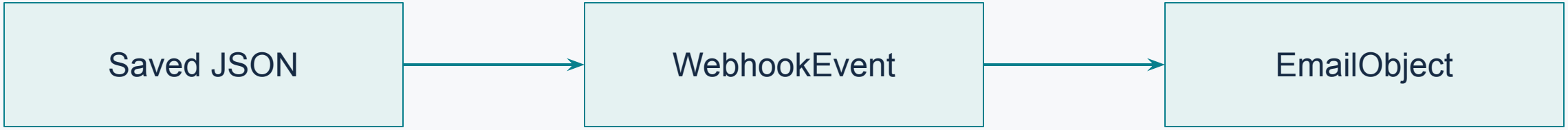
Step A: Data first

Capture real examples before writing logic

1. Delete the old webhook in the Nylas dashboard
2. Start Serveo and set SERVER_URL in .env
3. Run main.py, register the webhook, save WEBHOOK_SECRET
4. Restart main.py to load the new URL and secret
5. Send 3 examples and check requests/events

Code: app/config/config_webhook.py; app/main.py: webhook(), store_event_json()

Step B: Parse the saved JSON



load_email() reads the raw Nylas event

WebhookEvent validates the envelope

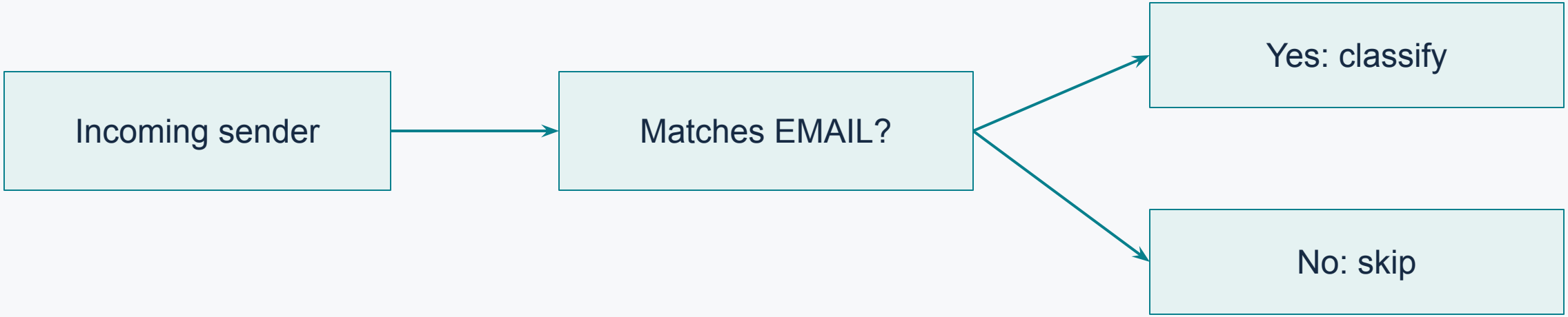
EmailObject validates the email inside data.object

The workflow uses typed sender, subject and body fields

Code: spam_workflow.py: load_email(); app/schemas/nylas_{webhook,email}_schema.py

Step C: Sender filter

is_from_me() compares the sender with EMAIL in .env



A testing guard for classification and deletion

Code: app/workflows/spam_workflow.py: is_from_me(); app/main.py: check_spam()

Step D: LLM classification

Goal: decide what the incoming email is

Python Enum: spam or not_spam

OpenAI structured output returns a Pydantic model

Classification contains category and confidence from 0 to 1

Same technique as the Week 2 structured_output example

Code: app/workflows/spam_workflow.py: EmailCategory, Classification, classify()

Step E: Action with confirmation

not_spam: no inbox action

spam: the page shows a Delete button

The user clicks Delete and confirms the browser prompt

The browser submits POST /delete/{email_id} to FastAPI

Confirmation in the page, action through FastAPI

Code: app/templates/index.html: form onsubmit; app/main.py: delete()

FastAPI: Already used in Week 2

FastAPI defines the API. Uvicorn runs the server.

FastAPI is a Python web framework that routes requests and validates data.

WEEK 2: ALREADY IN app/main.py

GET /events answers Nylas verification.
POST /events checks signatures and saves JSON.
GET / lists received emails.
FastAPI and Uvicorn: already in pyproject.toml.

WEEK 3: HANDLE CONFIRMED ACTIONS

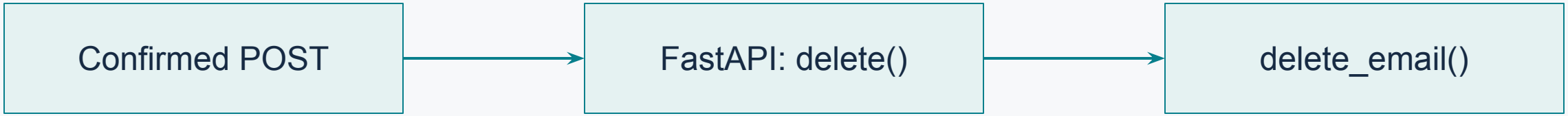
Add classification results to the existing page.
Add POST /delete/{email_id}: check spam, then call Nylas after user confirmation.

EXERCISE: week3/fastapi-tutorial

Stop the inbox app. In the tutorial folder, run uv run uvicorn main:app --reload.
Open localhost:8000/docs. Run uv run request.py in another terminal.

Code: app/main.py; pyproject.toml; tutorial: week3/fastapi-tutorial/README.md and request.py

Step F: FastAPI calls Nylas to delete



FastAPI checks the email and its spam classification

delete_email() creates a Nylas client from .env settings

```
nylas.messages.destroy(
    identifier=email.grant_id, message_id=email.id
)
```

Code: app/main.py: delete(); app/workflows/spam_workflow.py: delete_email()

Two ways to run it

SAVED EVENT IN THE TERMINAL

uv run app/workflows/spam_workflow.py <event-id>

Prompts y/N before deleting spam

AUTOMATICALLY ON NEW EMAIL

Run main.py and visit localhost:8000

Review the result, click Delete, then confirm

Code: app/workflows/spam_workflow.py: run(); app/main.py; app/templates/index.html

Known limits and next steps

CURRENT LIMITS

Results live in memory and reset on restart

The tunnel exposes the app. Protect /delete for long term use.

BEYOND SPAM

Build what you want to do with your email.
Add categories, extract information, or design another workflow.

NEXT WEEK: VECTOR DATABASE AND RAG

Store email events in a vector database for retrieval with RAG

Code: app/main.py: webhooks, classifications, delete(); roadmap: next week